Karger's Randomized Min-Cut Algorithm

Karger 随机最小割算法

1. Problem Setting / 问题背景

English. Let $G = (V, E)$ be an undirected, unweighted multigraph. A multigraph may contain parallel edges between the same pair of vertices, but self-loops are removed. Write $n = |V|$ and $m = |E|$.

中文。 设 $G = (V, E)$ 是一个无向、无权的多重图。多重图允许同一对顶点之间存在多条平行边，但自环会被删除。记 $n = |V|$、$m = |E|$。

English. A cut partitions V into two nonempty parts S and $V \setminus S$. Its cut-set is

中文。 一个割把 V 划分成两个非空部分 S 和 $V \setminus S$。对应的割边集合为

$$\delta(S) = \{uv \in E : u \in S,\ v \notin S\}.$$

English. Removing all edges in $\delta(S)$ disconnects the graph. The size of the cut is $|\delta(S)|$, where parallel edges are counted separately. A minimum cut is a cut of smallest size; denote its size by λ.

中文。 删除 $\delta(S)$ 中的所有边后，图会断开。割的大小是 $|\delta(S)|$，平行边必须分别计数。大小最小的割称为最小割，记其大小为 λ。

English. A deterministic solution can use repeated maximum-flow computations. Karger's algorithm takes another approach: it uses random edge contractions and accepts a small failure probability in exchange for a very simple algorithm.

中文。 确定性方法可以通过多次最大流计算求全局最小割。Karger 算法采用另一条路线：利用随机边收缩，并接受一个较小的失败概率，从而得到非常简洁的算法。

2. Edge Contraction / 边收缩

English. For an edge $e = uv$, contracting e means merging u and v into one supervertex. Edges incident to either endpoint are redirected to the new supervertex. Parallel edges are kept, while self-loops created by the merge are deleted. The resulting graph is denoted by G/e.

中文。 对边 $e = uv$ 进行收缩，就是把 u 和 v 合并成一个超级顶点。原来与两个端点相连的边都改为连接这个新顶点。平行边必须保留，合并后产生的自环则删除。收缩后的图记为 G/e。

English. Parallel edges represent different original edges and must all be counted in a cut. A self-loop never crosses a future cut because both endpoints are already inside the same supervertex.

中文。 平行边代表原图中不同的边，计算割大小时都要计数。自环的两个端点已经在同一个超级顶点中，所以它不可能跨越之后的任何割。

2.1 Adjacency-matrix implementation / 邻接矩阵实现

English. Let $A[u, w]$ be the number of parallel edges between u and w. When u and v are merged into x, for every other active vertex w set

中文。 令 $A[u, w]$ 表示 u 与 w 之间的平行边数量。把 u, v 合并成 x 时，对每个其他仍存在的顶点 w，令

$$A[x, w] = A[u, w] + A[v, w].$$

English. Set $A[x, x] = 0$ and mark one old row and column inactive. Only one row and one column are scanned, so a contraction costs $O(r)$ when r supervertices remain, and hence at most $O(n)$.

中文。 然后令 $A[x, x] = 0$，并把一个旧顶点对应的行和列标记为无效。由于只需扫描一行和一列，当还剩 r 个超级顶点时，一次收缩耗时 $O(r)$，因此最多为 $O(n)$。

English. If $k = A[u, v]$ edges join u and v, they become self-loops. The new degree is

中文。 如果 u, v 之间有 $k = A[u, v]$ 条边，它们会变成自环。新顶点的度数为

$$d(x) = d(u) + d(v) - 2k.$$

English. We subtract $2k$ because those edges were counted once in $d(u)$ and once in $d(v)$.

中文。 减去 $2k$ 是因为这些边在 $d(u)$ 和 $d(v)$ 中各被计算了一次。

---

3. Karger's Random Contraction Algorithm / Karger 随机收缩算法

English. Repeatedly choose an edge uniformly at random and contract it. When only two supervertices remain, all parallel edges between them form a cut of the original graph.

中文。 反复均匀随机选择一条边并收缩。当只剩两个超级顶点时，它们之间的所有平行边构成原图的一个割。

```
RandomContract(G)
    while G has more than 2 vertices:
        choose an edge e uniformly at random
        G <- G / e
    return all edges between the two remaining supervertices
```

```
随机收缩(G)
    当 G 中的顶点数大于 2 时:
        均匀随机选择一条边 e
        G <- G / e
    返回最后两个超级顶点之间的所有边
```

English. The algorithm always returns a valid cut, but not necessarily a minimum cut. For a fixed minimum cut C, it succeeds exactly when no edge of C is ever contracted.

中文。 算法一定返回一个合法的割，但不一定是最小割。对于一个固定最小割 C，当且仅当算法从未收缩 C 中的边时，该割才能被最终输出。

---

4. Choosing a Uniform Random Edge in $O(r)$ Time / 在 $O(r)$ 时间内均匀随机选边

English. Suppose the current graph has r supervertices and m edge copies. Store every degree $d(u)$, so that $\sum_u d(u) = 2m$. First choose a vertex u with probability

中文。 假设当前图有 r 个超级顶点和 m 条边副本。保存每个顶点的度数 $d(u)$，于是 $\sum_u d(u) = 2m$。首先按下面的概率选择顶点 u：

$$\Pr[u] = \frac{d(u)}{2m}.$$

English. Draw a uniform integer from $\{1, \dots, 2m\}$ and scan the prefix sums of the r stored degrees. This weighted choice costs $O(r)$.

中文。 从 $\{1, \dots, 2m\}$ 中均匀抽取一个整数，再扫描 r 个已保存度数的前缀和，即可完成加权选择，耗时 $O(r)$。

English. After choosing u, choose a neighbor v with conditional probability

中文。 选定 u 后，再按如下条件概率选择邻居 v：

$$\Pr[v \mid u] = \frac{A[u, v]}{d(u)}.$$

English. Scanning row u costs another $O(r)$. A specific edge copy $e = uv$ is selected through endpoint u with probability $d(u)/(2m) \cdot 1/d(u) = 1/(2m)$, and through v with the same probability. Thus

中文。 扫描邻接矩阵的第 u 行再花费 $O(r)$。一条具体边副本 $e = uv$ 通过端点 u 被选中的概率是 $d(u)/(2m) \cdot 1/d(u) = 1/(2m)$，通过端点 v 被选中的概率相同。因此

$$\Pr[\text{select } e] = \frac{1}{2m} + \frac{1}{2m} = \frac{1}{m}.$$

English. Every edge copy is therefore chosen uniformly. Edge selection costs $O(r)$, contraction costs $O(r)$, and one complete iteration still costs $O(r)$.

中文。 因而每条边副本都以相同概率被选中。随机选边耗时 $O(r)$，收缩也耗时 $O(r)$，所以一整轮仍为 $O(r)$。

4.1 Numerical example / 数值例子

English. Suppose AB has two parallel edges, AC has one edge, and BC has three parallel edges. Then $m = 6$ and $d(A) = 3, d(B) = 5, d(C) = 4$. The probability of choosing an AB edge is

中文。 假设 AB 有两条平行边，AC 有一条边，BC 有三条平行边。那么 $m = 6$，且 $d(A) = 3, d(B) = 5, d(C) = 4$。选中一条 AB 边的概率为

$$\frac{3}{12} \cdot \frac{2}{3} + \frac{5}{12} \cdot \frac{2}{5} = \frac{1}{3} = \frac{2}{6}.$$

English. This equals the number of AB edge copies divided by the total number of edge copies, as required.

中文。 这正好等于 AB 边副本数量除以总边数，符合均匀随机选边的要求。

5. Structural Lemmas / 结构性引理

Lemma 1: Contraction creates no new cuts / 引理 1：收缩不会创造新的割

English. Every cut of G/e corresponds to a cut of G. A supervertex represents a group of original vertices that must stay on the same side of the corresponding cut.

中文。 G/e 中的每一个割都对应于原图 G 的一个割。一个超级顶点代表一组必须放在同一侧的原始顶点。

Lemma 2: A fixed minimum cut survives a safe contraction / 引理 2：固定最小割在安全收缩后仍然存在

English. Let C be a fixed minimum cut of G and let $e \notin C$. The endpoints of e lie on the same side of C, so contracting e does not destroy C. Thus C remains a cut of G/e.

中文。 设 C 是 G 的一个固定最小割，并且 $e \notin C$。那么 e 的两个端点位于 C 的同一侧，收缩 e 不会破坏 C，所以 C 仍是 G/e 的一个割。

English. Every cut of G/e is also a cut of G, so G/e cannot contain a cut smaller than C. Since C still exists, it remains a minimum cut of G/e.

中文。 G/e 的每个割也都是 G 的割，所以 G/e 不可能出现比 C 更小的割。由于 C 仍然存在，它仍是 G/e 的最小割。

Lemma 3: Lower bound on the number of edges / 引理 3：当前边数的下界

English. If the current graph has r supervertices and minimum-cut size λ, every vertex has degree at least λ; otherwise, isolating a lower-degree vertex would give a smaller cut.

中文。 如果当前图有 r 个超级顶点，最小割大小为 λ，那么每个顶点的度数都至少为 λ；否则，把某个度数更小的顶点单独放在一侧就会得到更小的割。

English. By the handshake lemma,

中文。 根据握手定理，

$$2|E| = \sum_{v \in V} d(v) \geq r\lambda, \qquad \boxed{|E| \geq \frac{r\lambda}{2}}.$$

English. Here E is the current edge multiset, and parallel edge copies are counted separately.

中文。 这里的 E 是当前图的边多重集合，所有平行边副本都要分别计数。

6. Success Probability of One Run / 单次运行的成功概率

English. Fix a minimum cut C with $|C| = \lambda$. Conditional on C having survived so far, it is still a minimum cut. When r supervertices remain, $|E| \geq r\lambda/2$, so

中文。 固定一个最小割 C，其中 $|C| = \lambda$。在 C 此前一直存活的条件下，它仍然是当前图的最小割。当还剩 r 个超级顶点时，$|E| \geq r\lambda/2$，所以

$$\Pr[\text{next edge is in } C] = \frac{\lambda}{|E|} \le \frac{2}{r}.$$

English. Hence the next-step survival probability is at least

中文。 因而下一轮的存活概率至少为

$$\Pr[\text{survive next step}] \ge 1 - \frac{2}{r} = \frac{r-2}{r}.$$

6.1 Chain rule / 条件概率连乘

English. Let S_i be the event that the edge chosen in contraction i is not in C. The events need not be independent. By the chain rule,

中文。 令 S_i 表示第 i 次收缩选中的边不属于 C。这些事件不需要相互独立。根据链式法则，

$$\Pr\left[\bigcap_{i=1}^{n-2} S_i\right] = \prod_{i=1}^{n-2} \Pr\left[S_i \mid S_1 \cap \cdots \cap S_{i-1}\right].$$

English. Multiplying the per-step lower bounds gives

中文。 将每一步的下界相乘，得到

$$\begin{aligned}
\Pr[C \text{ survives to the end}] &\ge \frac{n-2}{n} \cdot \frac{n-3}{n-1} \cdots \frac{1}{3} \\
&= \prod_{r=3}^{n} \frac{r-2}{r} \\
&= \boxed{\frac{2}{n(n-1)}} \\
&> \frac{2}{n^2}.
\end{aligned}$$

English. This is a worst-case lower bound for returning one fixed minimum cut. The chance of returning some minimum cut may be larger if the graph has several minimum cuts.

中文。 这是返回某一个固定最小割的最坏情况概率下界。如果图中有多个最小割，返回任意一个最小割的概率可能更高。

English. For $n = 6$, the guaranteed probability is

中文。 当 $n = 6$ 时，保证的成功概率为

$$\frac{4}{6} \cdot \frac{3}{5} \cdot \frac{2}{4} \cdot \frac{1}{3} = \frac{1}{15} = \frac{2}{6 \cdot 5}.$$

---

7. A Concrete Graph Example / 一个完整图例

English. Consider two triangles. The left triangle has vertices A, B, C and edges AB, AC, BC; the right triangle has vertices D, E, F and edges DE, DF, EF. Add cross edges BE and CD.

中文。 考虑两个三角形。左侧三角形的顶点为 A, B, C，边为 AB, AC, BC；右侧三角形的顶点为 D, E, F，边为 DE, DF, EF。再加入两条跨边 BE 和 CD。

```
Left triangle / 左侧三角形       Right triangle / 右侧三角形

       A                                F
      / \                              / \
     B---C                            D---E

Cross edges / 跨边: B--E, C--D
```

English. The partition $\{A, B, C\} \mid \{D, E, F\}$ has cut-set $\{BE, CD\}$ of size two. No single edge disconnects the graph, so this is a minimum cut.

中文。 划分 $\{A, B, C\} \mid \{D, E, F\}$ 的割边集合是 $\{BE, CD\}$，大小为 2。删除任意一条边都不能使图断开，因此这是一个最小割。

English. A successful run may contract AB, then DF, and then continue merging only inside the two triangles. The final two supervertices are $\{A, B, C\}$ and $\{D, E, F\}$, with the two surviving cross edges between them.

中文。 一次成功运行可以先收缩 AB，再收缩 DF，然后只在两个三角形内部继续合并。最后两个超级顶点是 $\{A, B, C\}$ 和 $\{D, E, F\}$，它们之间保留两条跨边。

English. If BE or CD is contracted, this fixed cut is destroyed because vertices on opposite sides are merged. The algorithm might still find another minimum cut.

中文。 如果收缩 BE 或 CD，就会合并该固定割两侧的顶点，从而破坏这个割；但算法仍可能找到另一个最小割。

---

8. Probability Amplification / 用独立重复放大成功率

English. One run succeeds with probability at least $2/[n(n-1)] > 2/n^2$. If the algorithm is run independently R times and the smallest returned cut is kept, all runs fail with probability at most

中文。 一次运行的成功概率至少为 $2/[n(n-1)] > 2/n^2$。如果独立运行 R 次，并保留所有输出中最小的割，那么全部失败的概率至多为

$$\left(1 - \frac{2}{n^2}\right)^R.$$

English. Taking $R = tn^2$ and using $1 - x \le e^{-x}$ gives

中文。 令 $R = tn^2$，再利用 $1 - x \le e^{-x}$，得到

$$\Pr[\text{all runs fail}] \le \left(1 - \frac{2}{n^2}\right)^{tn^2} \le e^{-2t}.$$

English. Therefore the amplified algorithm returns a minimum cut with probability at least

中文。 因此，放大后的算法返回最小割的概率至少为

$$\boxed{1-e^{-2t}}.$$

English. Another common choice is to run $n(n-1)/2$ times. The success probability is then at least

中文。 另一种常见选择是运行 $n(n-1)/2$ 次，此时成功概率至少为

$$1-\left(1-\frac{2}{n(n-1)}\right)^{n(n-1)/2}\geq 1-\frac{1}{e}.$$

---

9. Running Time of Basic Karger / 基础 Karger 的运行时间

English. When r vertices remain, random edge selection and contraction cost $O(r)$. A complete run therefore costs

中文。 当还剩 r 个顶点时，随机选边和收缩共耗时 $O(r)$。因此一次完整运行耗时为

$$O(n)+O(n-1)+\cdots+O(3)=O(n^2).$$

English. If the basic algorithm is repeated tn^2 times as in the classroom amplification scheme, the total time is

中文。 如果按照课堂板书的放大方案，把基础算法重复 tn^2 次，那么总时间为

$$tn^2\cdot O(n^2)=\boxed{O(tn^4)}.$$

English. This method is simple, but the n^4 dependence motivates a faster divide-and-conquer version.

中文。 这种方法很简单，但 n^4 级别的代价促使我们设计更快的分治版本。

---

10. Contracting Only to t Vertices / 只收缩到 t 个顶点

English. Suppose random contraction stops when t supervertices remain. A fixed minimum cut must survive the contractions performed when the graph has $n, n-1, \ldots, t+1$ vertices.

中文。 假设随机收缩到剩余 t 个超级顶点时停止。一个固定最小割需要在图有 $n, n-1, \ldots, t+1$ 个顶点时进行的所有收缩中都存活。

English. Multiplying the conditional survival bounds gives

中文。 将各轮条件存活概率下界相乘，得到

$$\begin{aligned}\Pr[C \text{ survives down to } t \text{ vertices}] &\geq \prod_{r=t+1}^{n}\left(1-\frac{2}{r}\right)\\ &= \prod_{r=t+1}^{n}\frac{r-2}{r}\\ &= \frac{(t-1)t(t+1)\cdots(n-2)}{(t+1)(t+2)\cdots n}\\ &= \boxed{\frac{t(t-1)}{n(n-1)}}.\end{aligned}$$

English. For example, when $n=6$ and $t=4$,

中文。 例如，当 $n=6, t=4$ 时，

$$\frac{4}{6}\cdot\frac{3}{5}=\frac{4\cdot 3}{6\cdot 5}=\frac{2}{5}.$$

English. This is a guaranteed lower bound rather than the exact probability for every graph.

中文。 这是保证成立的下界，而不是每张图上的精确概率。

---

11. FastCut: Divide and Conquer / FastCut：分治加速

English. The most dangerous part of ordinary Karger contraction occurs near the end, when few vertices remain and the probability of hitting the minimum cut becomes large. FastCut stops earlier, creates two independent contracted graphs, and recursively solves both.

中文。 普通 Karger 收缩最危险的是后期：剩余顶点很少时，碰到最小割边的概率变大。FastCut 提前停止，独立生成两个收缩图，再分别递归求解。

```
FastCut(G)
    if |V(G)| <= 6:
        return an exact minimum cut by brute force

    t <- ceil(1 + |V(G)| / sqrt(2))
    G1 <- RandomContract(G, t)
    G2 <- RandomContract(G, t)   // independent of G1

    return the smaller of FastCut(G1) and FastCut(G2)
```

```
快速最小割(G)
    如果 |V(G)| <= 6:
        用暴力方法返回精确最小割

    t <- ceil(1 + |V(G)| / sqrt(2))
    G1 <- 随机收缩(G, t)
    G2 <- 随机收缩(G, t)        // 与 G1 独立
```

```
    返回 FastCut(G1) 与 FastCut(G2) 中较小的割
```

English. With $t = \lceil 1 + n/\sqrt{2} \rceil$, the survival probability of a fixed minimum cut in one branch is at least

中文。 当 $t = \lceil 1 + n/\sqrt{2} \rceil$ 时，一个固定最小割在单个分支中存活的概率至少为

$$\frac{t(t-1)}{n(n-1)} \geq \frac{1}{2}.$$

English. Since the two branches are generated independently, the probability that at least one branch preserves the cut is at least $1 - (1 - 1/2)^2 = 3/4$, before accounting for later recursive calls.

中文。 因为两个分支是独立生成的，所以暂不考虑后续递归时，至少一个分支保住该最小割的概率至少为 $1 - (1 - 1/2)^2 = 3/4$。

English. A full recursive analysis gives one-run success probability $\Omega(1/\log n)$. Repeating FastCut independently $O(\log n)$ times raises the success probability to high probability.

中文。 完整的递归分析可得，FastCut 单次成功概率为 $\Omega(1/\log n)$。再独立重复 $O(\log n)$ 次，就可以把成功率提高到高概率。

---

12. FastCut Running-Time Recurrence / FastCut 的运行时间递推式

English. Each call creates two recursive instances of size approximately $n/\sqrt{2}$. The contractions performed by the current call cost $O(n^2)$ in total, so

中文。 每次调用产生两个规模约为 $n/\sqrt{2}$ 的递归子问题，而当前调用中进行收缩的总成本为 $O(n^2)$，所以

$$\boxed{T(n) = 2T\left(\frac{n}{\sqrt{2}}\right) + O(n^2)}.$$

English. The exact algorithm uses $\lceil 1 + n/\sqrt{2} \rceil$. The ceiling and additive constant do not change the asymptotic result.

中文。 实际算法使用 $\lceil 1 + n/\sqrt{2} \rceil$。向上取整和加一不会改变渐近复杂度。

12.1 Work at one recursion level / 单层工作量

English. At depth k, there are 2^k subproblems, each of size approximately $n/(\sqrt{2})^k$. The total nonrecursive work at that level is

中文。 在深度 k 处有 2^k 个子问题，每个规模约为 $n/(\sqrt{2})^k$。该层非递归工作总量为

$$2^k \cdot O\left(\left(\frac{n}{(\sqrt{2})^k}\right)^2\right) = 2^k \cdot O\left(\frac{n^2}{2^k}\right) = O(n^2).$$

12.2 Why there are $O(\log n)$ levels / 为什么只有 $O(\log n)$ 层

English. Along a root-to-leaf path, after k levels the instance size is about $n/(\sqrt{2})^k$. The recursion stops when this is at most 6:

中文。 沿着一条从根到叶子的路径，经过 k 层后，问题规模约为 $n/(\sqrt{2})^k$。当它不超过 6 时停止：

$$\frac{n}{(\sqrt{2})^k} \le 6 \iff k \ge \log_{\sqrt{2}} \frac{n}{6} = 2\log_2 \frac{n}{6} = O(\log n).$$

English. Equivalently, every two levels reduce the problem size by half because $(1/\sqrt{2})^2 = 1/2$. Thus the recursion depth is $O(\log n)$.

中文。 等价地，因为 $(1/\sqrt{2})^2 = 1/2$，每递归两层，问题规模大约减半，所以递归深度为 $O(\log n)$。

English. Each level costs $O(n^2)$ and there are $O(\log n)$ levels; therefore

中文。 每一层花费 $O(n^2)$，共有 $O(\log n)$ 层；因此

$$\boxed{T(n) = O(n^2 \log n)}.$$

English. Repeating FastCut $O(\log n)$ times for high success probability gives total time

中文。 为得到高成功率，再把 FastCut 重复 $O(\log n)$ 次，总时间为

$$\boxed{O(n^2 \log^2 n)}.$$

---

13. Corollary: Number of Minimum Cuts / 推论：最小割的数量

English. Suppose a graph has M distinct minimum cuts. Karger's algorithm outputs each fixed minimum cut with probability at least $2/[n(n-1)]$. The output events for different cuts are mutually exclusive, so

中文。 假设一张图有 M 个不同的最小割。Karger 算法输出每个固定最小割的概率至少为 $2/[n(n-1)]$。输出不同割的事件彼此互斥，所以

$$M \cdot \frac{2}{n(n-1)} \le 1, \qquad \boxed{M \le \frac{n(n-1)}{2}}.$$

English. This is an example of the probabilistic method: the conclusion is deterministic, but the proof uses a randomized algorithm. The bound is tight for an n-vertex cycle. The old webpage writes $n(n-2)/2$ in one heading; its proof and the correct bound are $n(n-1)/2$.

中文。 这是概率法的一个例子：结论本身是确定性的，但证明使用了随机算法。对于 n 个顶点的环，这个上界可以达到。旧网页的某个标题写成了 $n(n-2)/2$；其证明以及正确结论都是 $n(n-1)/2$。

---

14. Common Points of Confusion / 常见易错点

English. "Uniformly choose an edge" means uniform over edge copies, not uniform over adjacent vertex pairs. Three parallel edges must make that pair three times as likely to be selected as a pair joined by one edge.

中文。 "均匀随机选择一条边"是对边副本均匀，而不是对相邻顶点对均匀。三条平行边会使该顶点对被选中的概率变成只有一条边时的三倍。

English. Contracting an edge of one fixed minimum cut destroys that cut, but does not necessarily make the whole run fail; another minimum cut may survive.

中文。 收缩某个固定最小割中的边会破坏该割，但不一定意味着整次运行失败；另一个最小割仍可能存活。

English. The product of survival probabilities uses conditional probabilities. Independence between contraction steps is not assumed.

中文。 存活概率的乘积使用条件概率，并没有假设各轮收缩事件相互独立。

English. The bounds $2/[n(n-1)]$ and $t(t-1)/[n(n-1)]$ are guaranteed lower bounds, not exact probabilities for every graph.

中文。 $2/[n(n-1)]$ 和 $t(t-1)/[n(n-1)]$ 都是保证成立的下界，不是每张图上的精确概率。

English. In $T(n) = 2T(n/\sqrt{2}) + O(n^2)$, the factor 2 controls the width of the recursion tree. Repeated shrinking by $1/\sqrt{2}$ controls its $O(\log n)$ depth.

中文。 在 $T(n) = 2T(n/\sqrt{2}) + O(n^2)$ 中，系数 2 决定递归树的宽度；每层乘以 $1/\sqrt{2}$ 的规模缩减决定其 $O(\log n)$ 深度。

---

15. Formula Sheet / 公式速查

English	中文	Formula
Minimum-cut size	最小割大小	$\lambda=
Handshake lemma	握手定理	$\sum_v d(v)=2
Edge-count lower bound	剩余 r 个点时的边数下界	$
Probability of hitting C	下一轮碰到固定最小割	$\le 2/r$
One-step survival	下一轮存活概率	$\ge 1-2/r$
One-run success	单次成功概率	$\ge 2/[n(n-1)]$
Survival down to t vertices	收缩到 t 点后的存活概率	$\ge t(t-1)/[n(n-1)]$
One basic run	基础算法单次时间	$O(n^2)$
tn^2 repetitions	基础算法重复 tn^2 次	$O(tn^4)$
Failure after tn^2 runs	重复后的失败概率	$\le e^{-2t}$
FastCut recurrence	FastCut 递推式	$T(n) = 2T(n/\sqrt{2}) + O(n^2)$
One FastCut run	FastCut 单次时间	$O(n^2 \log n)$
High-probability FastCut	高概率 FastCut	$O(n^2 \log^2 n)$

---

16. One-Sentence Summary / 一句话总结

English. Karger's algorithm repeatedly merges randomly chosen adjacent groups; a fixed minimum cut is found whenever none of its edges is contracted, and independent repetition or divide-and-conquer turns this simple idea into an effective randomized min-cut algorithm.

中文。 Karger 算法不断随机合并相邻顶点组；只要某个固定最小割中的边从未被收缩，该割就会被找到，而独立重复或分治递归可以把这一简单思想变成有效的随机最小割算法。

References / 参考资料

- Randomized Algorithms (Fall 2015) course page
- Karger's Min-Cut lecture note
- Classroom board photographs supplied with these notes / 本笔记所附课堂板书照片